Unified Engineering Workbench

Complete assignment submission (PDF edition)

Candidate ID (roll number): RA2311056010232

Date: 01-11-2026

Where the full project package is

The submission form accepts only a PDF of at most 1 MB, so the complete package required by section 8 of the brief is hosted as a ZIP (about 13 MB, named CandidateID_Workbench.zip). It contains Launch.exe and the packaged application, src/ (frontend and C++ backend), CMakeLists.txt with dependency lock, editor run configuration, data/ with provenance, tests/, results/ (raw CSV and labelled plots), docs/, README, the Git bundle and readable commit log, and the AI-use log.

Download the ZIP:
https://github.com/FarazAhmed0928/workbench/releases/download/latest-build/CandidateID_Workbench.zip

Source repository: https://github.com/FarazAhmed0928/workbench

Unzip with "Extract All" and double-click Launch.exe. No installer, compiler, terminal or internet connection is needed. Windows may show "Windows protected your PC" because the executable is unsigned: choose More info, then Run anyway.

If either link is unavailable, please contact me through SRM Career Centre and I will supply the ZIP by another route.

What this PDF contains

1. Technical report (design, FEA verification, performance, embedded design, backpropagation, tests, limitations). 2. Handwritten backpropagation appendix (2 pages). 3. Task E design reference (interface table and plan). 4. Clean-machine launch evidence from a Windows 11 virtual machine. 5. AI-use log. 6. Ownership statement.

Headline results

Item	Result
Thermal FEA, N = 10/20/40/80	Max midpoint error 1.25, 0.3125, 0.0781, 0.0195 C; observed order 2.00
Backpropagation check	Max gradient error about 1e-10 (limit 1e-5)
Benchmark (Apple M4, 2 runs)	4 workers: 3.51x and 3.67x; serial/parallel difference 0
Tests	5 CTest suites pass on Linux, Windows (CI) and macOS
Clean-machine launch	Windows 11 ARM VM: server ready 6 ms, UI loaded 429 ms; exits when the tab closes

Unified Engineering Workbench: Technical Report

Candidate ID: RA2311056010232

1. Design choices

The backend is C++17 and the front end is a browser page served from the same executable. Core logic (FEA, parsers, thread pool, batch, backprop, mock devices) lives in a static library, wb_core, with no HTTP or UI code. A thin server layer maps JSON requests onto it. Launch.exe starts the server on 127.0.0.1 only, on a free port, opens the default browser and exits when the page stops sending heartbeats, so no terminal, install or firewall prompt is needed. MSVC builds use the static runtime, and CI checks that Launch.exe imports no VC++ runtime DLL. Three.js and Chart.js are bundled locally, so the app works offline.

Two inputs are supported. An ASCII STL is parsed and drawn as a rotatable, pannable, zoomable surface (units are model units because STL has no unit field). A nodal CSV with the header node_id,x_m,y_m,z_m,temperature_C (metres, degrees C) is drawn as a colour-mapped point cloud with a legend; clicking a node shows its id, coordinates and temperature. Malformed files are rejected with a line-numbered message and never crash the server. The bundled examples are synthetic and labelled as such in data/README.md.

2. Thermal FEA verification

Two-node linear elements assemble K = (kA/h)[[1,-1],[-1,1]] and a consistent load qAh/2 per node. Dirichlet values are moved to the right-hand side and the constrained rows become identity rows. The error is measured at element midpoints from the FE interpolation, not only at nodes.

N	h [m]	Nodal error [C]	Midpoint error [C]	Order
10	0.1	2.8e-14	1.25	-
20	0.05	2.0e-13	0.3125	2.00
40	0.025	8.8e-13	0.0781	2.00
80	0.0125	1.2e-12	0.0195	2.00

The midpoint error equals (q/2k)h²/4 exactly, which a unit test checks. Debugging insight: nodal values are exact to round-off for a quadratic solution, so a nodes-only check would hide the discretisation error entirely; the h² behaviour only appears between nodes.

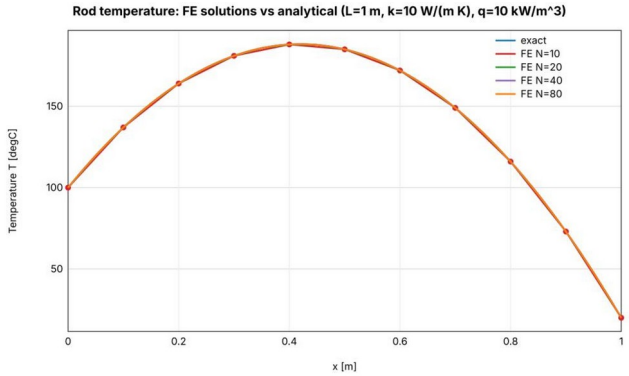


Fig. 1a: FE vs exact, N = 10 to 80.

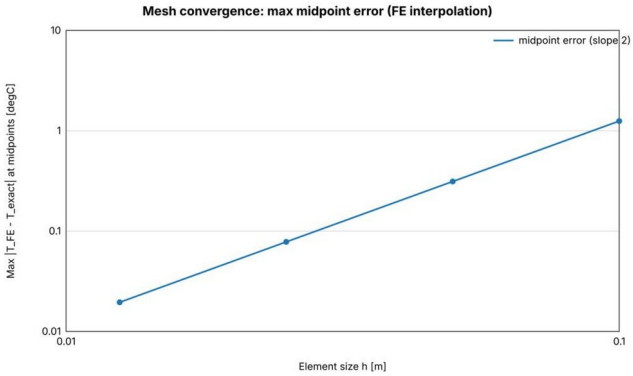


Fig. 1b: midpoint error vs h.

3. Performance results

Workload: 20,000 independent solves of 2,000 elements, seed 12345, results kept in input order. A bounded std::thread pool is created outside the timed

region. Protocol: one warm-up, five measured runs, median reported; only the solve is timed. Machine: Apple M4 (10 cores, 10 logical), 16 GiB, macOS, Apple Clang 17, -O3 -DNDEBUG, a passively cooled MacBook Air.

Workers	Run 1 [s]	Run 2 [s]	Speedup	Efficiency
1	0.2348	0.2453	1.00	1.00
2	0.1242	0.1234	1.89 / 1.99	0.95 / 0.99
4	0.0668	0.0668	3.51 / 3.67	0.88 / 0.92

Serial and parallel temperatures agree exactly (maximum difference 0, limit 1e-8 C). The parallel medians repeat within 1% between runs, while the serial median moved by 4% (individual serial runs ranged from 0.228 to 0.275 s), so most of the speedup difference between runs comes from the serial baseline. Efficiency falls at 4 workers because of thread start-up and queueing, memory bandwidth, and a short run (about 0.07 s) where small noise matters. Throttling is possible but was not measured. The Windows build was only tested in a virtual machine, so no Windows timings are claimed. Launch-to-ready time is reported separately in the app footer (Windows VM: server ready 6 ms and UI loaded 429 ms after process start, in a Windows 11 ARM virtual machine with 4 logical CPUs running the x64 build under emulation).

4. Embedded design

The design (docs/embedded_design.md) separates three voltage domains for the microcontroller, FPGA and JTAG adapter. The adapter drives TCK, TMS and TDI, the target drives TDO, and VTref is sensed, not supplied. A JTAG adapter is a debug interface: it runs no application and its telemetry describes the link. No shared connector, voltage or protocol is assumed. The C++ mock adapters demonstrate discovery, capabilities, connect, disconnect, telemetry and a recoverable timeout with retry; everything is labelled SIMULATED.

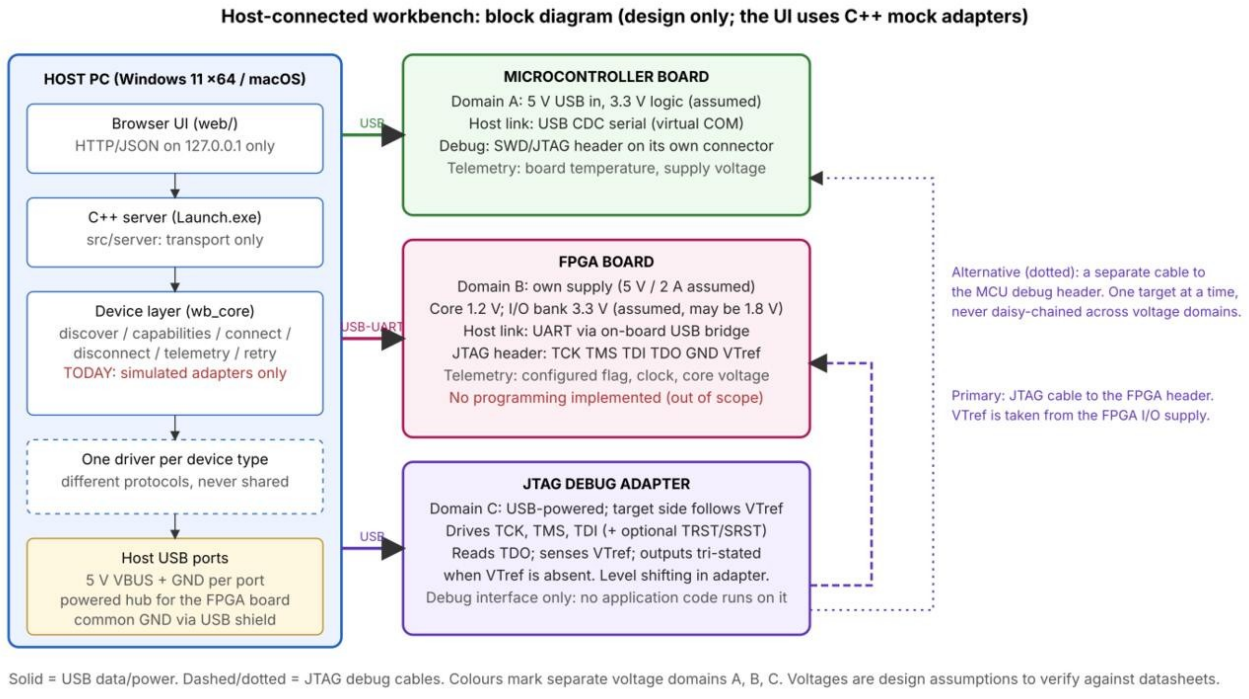


Fig. 2: block diagram (design only).

5. Backpropagation

With x=1, y=2, w=0.5, b=0, v=1, c=0 and learning rate 0.1: z=0.5, h=0.4621, y_hat=0.4621, J=1.1825. The gradients are dJ/dv=-0.7107, dJ/dc=-1.5379, dJ/dw=dJ/db=-1.2095, all negative, so every parameter increases and the prediction moves toward the target. After one simultaneous update J=0.6858. Central differences (epsilon 1e-6) agree with the analytic gradients to 1.1e-10 (limit 1e-5). Over 100 steps the loss falls monotonically to 1.7e-20 (Fig. 3); one sample can be fitted exactly, so this shows correct descent, not generalisation.

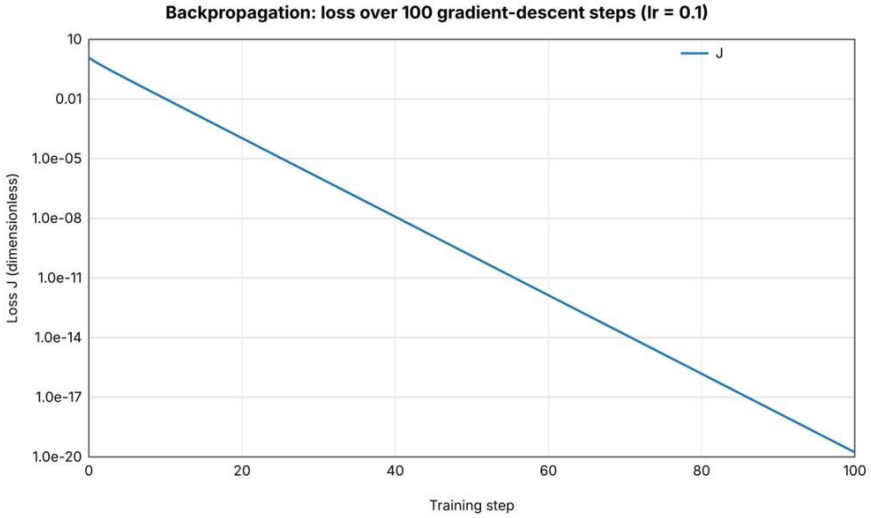


Fig. 3: loss over 100 gradient-descent steps (log scale).

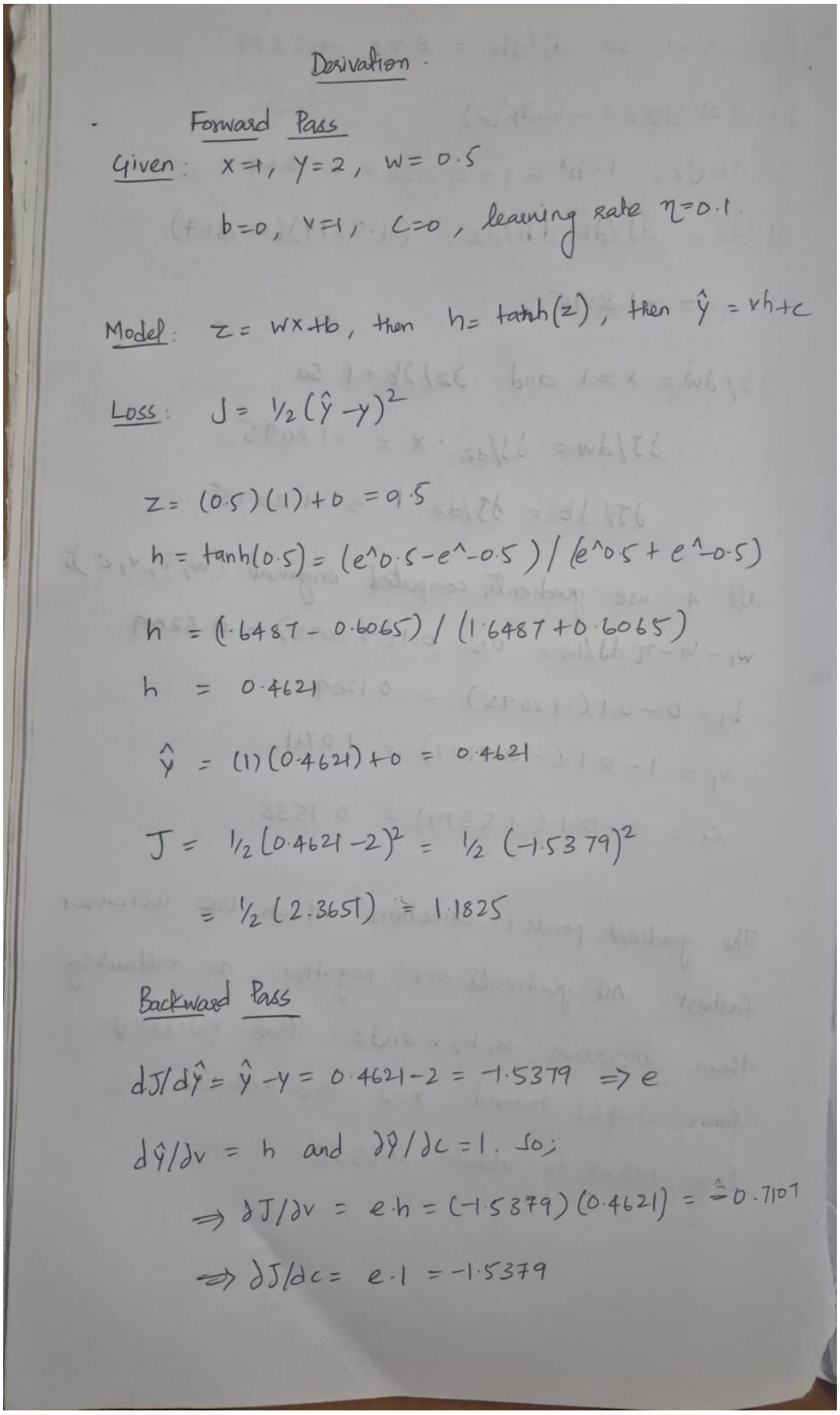
6. Test outcomes

- Five CTest suites (FEA, parsers, batch, backprop, devices) pass on Linux (GCC 13), Windows (MSVC, in CI) and macOS (Apple Clang 17).
- Negative tests: empty, truncated, binary, non-finite and malformed STL/CSV, invalid FE inputs, cancellation, device errors. A ThreadSanitizer run found no data races.
- Packaged Launch.exe was smoke-tested in CI and launched by double-click on a Windows 11 ARM virtual machine (x64 emulation).
- Tested limits: ASCII STL up to 190,000 triangles (45 MB) and CSV up to 1,000,000 rows (40 MB) accepted; larger files are rejected with a message.

7. Limitations

- Parts of the code were written with AI assistance (docs/ai_use_log.md); I reviewed and tested it and can explain it.
- Hardware is simulated; the voltage table is a set of assumptions, not a verified design.
- 1D, constant-property, steady-state thermal model; ASCII STL only; example data are synthetic, not ANSYS exports.
- Launch.exe is unsigned, so SmartScreen may warn. Browser rendering time for the largest files and Windows benchmark timings were not measured.

Appendix: handwritten backpropagation (page 1 of 2)



Derivation

Forward Pass

Given: $x=1,\ y=2,\ w=0.5$

$b=0,\ v=1,\ c=0$, learning rate $\eta=0.1$

Model: $z = wx+b$, then $h=\tanh(z)$, then $\hat{y} = vh+c$

Loss: $J = \frac{1}{2}(\hat{y}-y)^2$

$z = (0.5)(1)+0 = 0.5$

$h = \tanh(0.5) = (e^{0.5}-e^{-0.5})/(e^{0.5}+e^{-0.5})$

$h = (1.6487 - 0.6065)/(1.6487+0.6065)$

$h = 0.4621$

$\hat{y} = (1)(0.4621)+0 = 0.4621$

$J = \frac{1}{2}(0.4621-2)^2 = \frac{1}{2}(-1.5379)^2$

$= \frac{1}{2}(2.3651) = 1.1825$

Backward Pass

$dJ/d\hat{y} = \hat{y}-y = 0.4621-2 = -1.5379 \Rightarrow e$

$d\hat{y}/dv = h$ and $\partial\hat{y}/\partial c = 1$. So;

$\Rightarrow \partial J/\partial v = e\cdot h = (-1.5379)(0.4621) \doteq -0.7107$

$\Rightarrow \partial J/\partial c = e\cdot 1 = -1.5379$

Appendix: handwritten backpropagation (page 2 of 2)

$\partial \hat{y} / \partial h = v = 1$, so $dJ/dh = e \cdot v = +1.5379$

$d \tanh(z)/dz = 1 - \tanh^2(z)$,

So $dh/dz = 1 - h^2 = 1 - 0.2136 = 0.7864$

$dJ/dz = dJ/dh \cdot dh/dz = (+1.5379)(0.7864)$

$= -1.2095$

$\partial z / \partial w = x = 1$ and $\partial z / \partial b = 1$ so.

$\partial J / dw = \partial J / dz \cdot x = -1.2095$

$\partial J / \partial b = \partial J / dz \cdot 1 = -1.2095$

all 4 use gradients computed original (w, b, v, c);

$w_1 = w - \eta \cdot dJ/dw = 0.5 - 0.1(+1.2095) = 0.6209$

$b_1 = 0 - 0.1(+1.2095) = 0.1209$

$v_1 = 1 - 0.1(-0.7107) = 1.0711$

$c_1 = 0 - 0.1(+1.5379) = 0.1538$

The gradient points in direction which loss increases fastest. All gradients are negative, so subtracting them increases w, b, v and c. That raises $\hat{y}$ from 0.461 toward $y = 2$ and reduces loss from 1.1825 to about 0.6858.

Task E: Embedded-system integration design

Scope: a design for a host-connected workbench with **one microcontroller board, one FPGA board and one JTAG debug adapter**. Only the host-side discovery/connect/telemetry/recovery logic is implemented, as **C++ mock adapters** (`src/core/devices.*`). Nothing in this repository talks to real hardware, and all UI output for this view is labelled SIMULATED.

Interface and voltage table

All voltages are **assumptions for a typical hobby/evaluation setup**, not properties of any specific product. Each real board must be checked against its own datasheet before connecting anything.

Item	Microcontroller board	FPGA board	JTAG adapter
Role	Runs firmware; sensor/control node	Programmable-logic target	Debug interface to a target's TAP; does not compute
Power in	5 V from host USB (VBUS)	Own 5 V / 2 A supply (not from a bus-powered port)	5 V from host USB (VBUS)
Voltage domain	A: 3.3 V logic	B: core 1.2 V, I/O bank 3.3 V (may be 1.8 V or 2.5 V)	C: adapter core 3.3 V; **target-side I/O follows VTref**
Ground	Common GND to host via USB shield	Common GND via the adapter's GND pins and USB shield	GND pin on JTAG cable is the reference for all JTAG signals
Host communication	USB CDC-ACM (virtual COM), line-framed telemetry	UART through the board's USB-UART bridge	USB with a vendor/probe protocol (e.g. MPSSE or CMSIS-DAP style); not a serial console
Debug connector	SWD/JTAG header (its own pin-out)	JTAG header: TCK, TMS, TDI, TDO, GND, VTref	Target cable per target; **no assumed common connector**
Level shifting	None needed at 3.3 V host-side; add a buffer if a 5 V peripheral is attached	Required if bank voltage is not 3.3 V	Built in; output buffers powered from VTref
Pinout assumption	Documented per board in a device	Documented per board in a device	Pin-out per target cable; profile selects

Item	Microcontroller board	FPGA board	JTAG adapter
	profile	profile	it

JTAG signal ownership

Signal	Driven by	Notes
TCK, TMS, TDI	Adapter	Series resistors (about 22 to 33 ohm) close to the adapter limit ringing and fault current
TDO	Target	The adapter only receives it
TRST / SRST	Adapter (optional)	Open-drain preferred; use only if the target exposes them
VTref	Target	Target I/O supply, **sensed** by the adapter. The adapter does not power the target through it
GND	Shared	At least one GND per signal group on the cable

Why a JTAG adapter is not an interchangeable board

A JTAG adapter has no application processor, no firmware the user writes and no sensor values of its own. Its job is to move bit patterns on TCK/TMS/TDI/TDO into a target's test-access port. Its capabilities (chain scan, TCK rate, VTref sense) are properties of the **link**, not of a compute node. In the software model it therefore reports different capabilities and different telemetry (`target_vref`, `tck`, `devices_in_chain`) from the MCU and FPGA, and it needs a target to be meaningful. Boards also differ in connector, voltage and protocol, so each device type has its **own driver** behind a common interface (`discover`, `capabilities`, `connect`, `disconnect`, `readTelemetry`), and nothing assumes a shared connector, voltage or protocol.

Safe integration plan

1. **Paper review first.** Read each board's datasheet for I/O voltage, absolute maximum ratings and JTAG pin-out. Fill in the device profile.
2. **Bench power only.** Power each board from its own supply with a current limit, with no signal cables attached. Measure the rails (3.3 V, core, VTref) before any connection.
3. **Common ground, then signals.** Connect ground first. Add USB data cables, one board at a time, and confirm enumeration and telemetry.
4. **JTAG last, one target at a time.** Check VTref on the target header. Connect the adapter to the FPGA only. Run a chain scan and confirm the expected IDCODE. Only afterwards consider the MCU debug header, with a separate cable.
5. **Never daisy-chain across domains.** The MCU and FPGA are not placed on one JTAG chain unless they share the same I/O voltage and the target documentation allows it.

6. **Fail safe.** The adapter tri-states its outputs when VTref is absent. The host treats a missing device, a timeout or an unexpected voltage as an error, and does not retry writes blindly.
7. **ESD and handling.** Use an earthed wrist strap, avoid hot-plugging JTAG cables on a powered target, and keep wires short.

What the mock adapters demonstrate

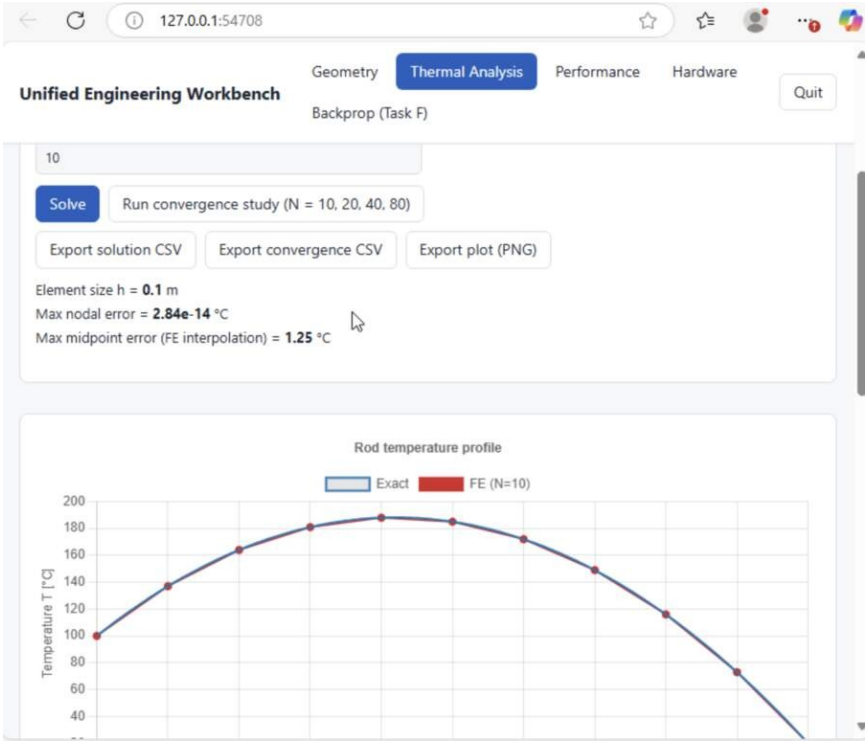
- Discovery of all three device types and capability reporting.
- Connect and disconnect with explicit errors (not found, already connected, not connected).
- One telemetry message per device type, with a sequence number and `simulated: true`.
- **One recoverable failure:** "Simulate timeout" makes the next connect or telemetry call time out. "Connect with retry" retries up to 3 times and reports the number of attempts; a telemetry timeout leaves the device connected, so the next read succeeds. If retries are exhausted, it gives up with a clear message instead of looping.
- Unit tests: `tests/test_devices.cpp`.

Limitations

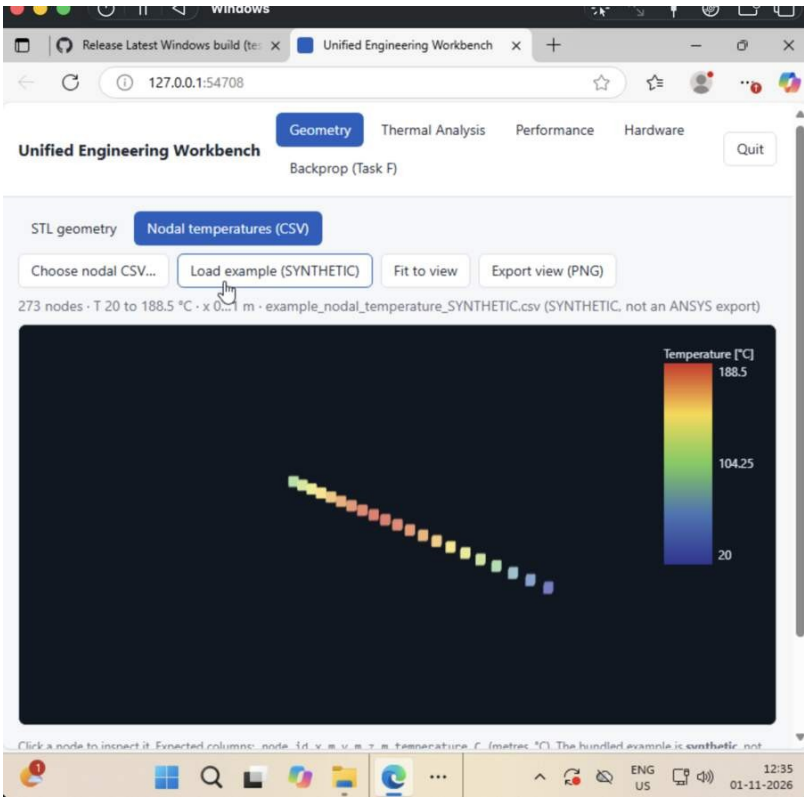
- No real I/O, USB enumeration, serial framing, JTAG bit-banging or FPGA programming. These are explicitly out of scope.
- Telemetry values are synthetic (smooth functions of a counter).
- The voltage table is an assumption set, not a verified design. A real build needs datasheet checks, a power-budget calculation, and review of ESD and layout.
- Retry back-off is simulated (not slept), and there is no concurrency control beyond a mutex.

Clean-machine launch evidence

Windows 11 Pro 24H2 (build 26100) ARM64 virtual machine on an Apple M4 Mac, with no compiler or developer tools installed. The x64 Launch.exe ran under Windows x64 emulation. Double-clicking Launch.exe opened the default browser with the workbench; closing the tab ended the Launch.exe process within 25 seconds (checked in Task Manager). Startup: server ready in 6 ms, UI loaded in 429 ms after process start.



Thermal Analysis in the Windows VM (units shown, midpoint error 1.25 C for N = 10).



Synthetic nodal-temperature CSV in the Windows VM, with colour legend in degrees C.

AI-use log

Prepared from the real working session and reviewed by the candidate on 01-11-2026. Nothing here is invented.

Tool: Claude (Anthropic), via Claude Code / claude.ai session, model reported as `claude-sonnet-5-5` (the serving model may differ). Commits made by the assistant carry a `Co-Authored-By: Claude` trailer.

1. Representative prompts and outputs

#	My prompt (verbatim or close)	What the AI produced	What I did with it
1	Attached the brief; "plan how can we do the project, state the requirements which I would need to bring in order to complete this project successfully with your help"	A plan: C++17 backend as a local HTTP server, web UI, `Launch.exe` launcher; a 15-day schedule; a list of what only I can do (handwritten backprop, VM launch test, honest AI log)	Accepted the architecture; the schedule had to be replaced (see 2a)
2	"is it required to have an exe file rather I can show it as a website result"	Cited the brief's one-click launch requirement and shortlisting gate; a hosted website alone cannot satisfy it	Checked the wording in the brief myself; kept Launch.exe
3	"I received it on 31/10/2026, submission is in 2 days" (with the forwarded mail)	A compressed ~25-hour schedule, plus a way to build and test Windows output from a Mac (GitHub Actions + Windows 11 ARM VM)	Accepted; set up GitHub repo and UTM VM
4	"can it be because repo is private because asked it to be private"	Explained the connector could not see private repos not granted to it; suggested making the repo public during development	Made the repo public temporarily
5	"can't we produce real data ANSYS	Declined to fabricate an "ANSYS export";	Chose to keep synthetic data,

#	My prompt (verbatim or close)	What the AI produced	What I did with it
	export”	pointed out the brief allows a self-created synthetic CSV if clearly labelled; offered to use a real export if I supplied one	labelled as such
6	“continue with synthetic and continue your task”	Built Task B parsers, Task D pool and benchmark, Task F backprop, Task E mocks, server and UI, with tests	Reviewing the code; see section 4
7	“launch.exe is working but I don’t have data”	Explained the app starts empty and the examples load from buttons; I then confirmed the `data/` folder was present and the examples worked	Confirmed on the Windows VM

2. Incorrect or unsuitable suggestions I identified

a. **Schedule based on a 15-day timeline.** The first plan assumed the brief’s nominal duration. My real deadline was about 41 hours away (mail: 02-11-2026, 10:00 AM). I supplied the deadline and the plan was cut down.

b. **Wrong test expectation (caught by the test suite).** The assistant wrote a parser test expecting the synthetic CSV’s maximum temperature to be 100.5 °C. The data actually peaks near 188.5 °C (the rod profile’s maximum), so the test failed. The assistant had assumed the maximum was at $x = 0$ and had not checked against the profile $100 - 80x + 500x(1 - x)$. The test and `data/README.md` were corrected.

c. **“UTM’s built-in Windows guide” does not include the Windows ISO.** I asked to avoid external downloads; the assistant corrected itself and said a separate ISO download (CrystalFetch) is unavoidable.

d. **GitHub “Artifacts” download.** The assistant told me to click the artifact in the Actions page; as a signed-out user in the VM I had nothing to click (artifacts need a GitHub login). The workflow was changed to also publish a plain-download pre-release for testing.

e. **First Windows CI run failed** because the packaging step wrote into `docs/` before it existed (the assistant’s script bug). Found from the job-step status, fixed and re-run; the second run was green.

f. **First block diagram had overlapping connector lines and text overflowing its boxes.** I looked at the rendered image (not just the SVG source), saw the problems, and the diagram was redrawn.

g. **Large-upload test returned HTTP 413 for every file** when sent with curl's default content type. With `Content-Type: text/plain` (what the browser's `fetch` with a string body sends) the same files were accepted. The cause was not investigated in the library source; I treat it as unverified. Resolved later: the 45 MB STL and 40 MB CSV were loaded through the real browser UI in headless Chrome without errors (about 3.5 s each).

3. Verification evidence (what I ran, not what I was told)

- `ctest`: FEA (BCs, nodal and midpoint errors, order 2.00), parsers (positive and negative), batch (serial = parallel, order, cancel), backprop (gradient check ~1e-10 vs 1e-5 tolerance), devices (errors and recovery).
- ThreadSanitizer build of the batch tests: no data races reported.
- Headless-Chrome run of every UI view, including node inspection, malformed-file banners, form validation, benchmark and device recovery.
- GitHub Actions on Windows: MSVC build, tests, static-runtime check, packaged-`Launch.exe` smoke test (green).
- Launch on a clean Windows 11 ARM VM (x64 emulation): `Launch.exe` opens the app, examples load; closing the tab made `Launch.exe` disappear from Task Manager within 25 s. Screenshots and startup timing (server ready 6 ms, UI loaded 429 ms) are in `docs/evidence/`.
- Built and tested on my MacBook Air (Apple M4, Apple Clang 17): all 5 test suites passed. Two benchmark runs are in `results/benchmark_run1.csv` and `benchmark_run2.csv` (4-worker speedup 3.51 and 3.67; serial/parallel difference 0).
- Handwritten backprop pages: I wrote them by hand. The assistant supplied a line-by-line worked outline when I asked what to write, so the structure and wording follow it; I then compared my numbers with `results/backprop_report.txt` (the C++ output) and the assistant's review of my photo pointed out minus signs that look like plus signs and page show-through. I chose to submit the pages as photographed, without retaking them.

4. My edits and decisions

- Chose synthetic example data and its labelling.
- Chose the VM-based launch test and installed UTM and Windows 11 ARM myself.
- Chose to make the repository public temporarily and will remove the test release before submission.
- Ran the Mac build and benchmark myself, and made a real commit of the results from my machine.
- Decided to anonymise the git author (history rewritten once, tree identical) and to use my roll number RA2311056010232 as the candidate ID.
- Code I have read through and can explain: FEA assembly and boundary conditions (`fea1d.cpp`), the midpoint-error calculation, the backprop chain rule

(`backprop.cpp` and my handwritten pages), and how the thread pool keeps results in input order (`thread_pool.cpp`, `batch.cpp`).

5. Deliberate challenge test

Not needed: section 2 documents real incorrect or unsuitable suggestions that I identified.

Ownership statement

Reused third-party code (unmodified, vendored with licence and version): - cpp-httplib 0.18.3 (MIT) and nlohmann/json 3.11.3 (MIT), in `third_party/`. - Three.js 0.160.1 (MIT) with `OrbitControls`, and Chart.js 4.4.1 (MIT), in `web/vendor/`. - C++ standard library, CMake, and the GitHub Actions `checkout` and `upload-artifact` actions.

Original work in this repository: the FEA solver, parsers, thread pool, batch generator, benchmark harness, backprop code, mock device adapters, SVG plot writer, HTTP API, launcher, web UI, tests, example-data generator, documents and CI workflow.

AI assistance: The source code, tests, CI file and drafts of the documents were written with Claude (Anthropic) in an interactive session, under my direction. I reviewed, built, ran and tested them, and I take responsibility for them. The handwritten backpropagation pages are my own work. Prompts, outputs, mistakes found and my edits are in `docs/ai_use_log.md`.

Data: both example files are self-generated and synthetic (`data/README.md`); no ANSYS or third-party data is included.

Candidate ID (roll number): RA2311056010232 Date: 01-11-2026